The second thing you'll need to do to try the new features on the CoffeeScript REPL is to pass arguments to Node.js when you start the CoffeeScript REPL telling it to enable the new features. When invoking the CoffeeScript REPL, these options are passed individually as flags. For example, to enable proxies you set the `harmony-proxy` flag:

```
> coffee -i --nodejs --harmony-proxies
```

What if you want to use all of the new features? Well, Harmony was the name given early on to a large set of features that were intended for the next version of the ECMAScript specification. You can enable all of these Harmony features in CoffeeScript/node.js/V8 with a single `harmony` flag:

```
> coffee -i --nodejs --harmony
```

To try the CoffeeScript code snippets for the features in this section, you should invoke the REPL with `harmony` enabled.

The first feature worth mentioning—in part because it's a point of contention—is the module system.

13.3.1. Modules

JavaScript wasn't designed with a module system, so when people needed modules for client-side programs they invented their own. Now there are many different module systems, so almost everybody is a traveler in a foreign country—carrying their module adapters everywhere they go so they can plug things in.

Until ECMAScript 6 standardizes modules, you'll need to use some other module system. One approach, outlined in chapter 12, is to use the Node.js module syntax for both server and client modules and compile them out for the client (where they aren't supported) using a build step.

In the long term, JavaScript needs a standardized module system for the sake of interoperability. The module system that's proposed for ECMAScript 6 is similar to the Node.js module system. But the keyword